

Islamabad PM2.5 Air Quality Forecasting System

An End-to-End MLOps Platform for Hourly
PM_{2.5} Air-Quality Forecasting

Author: Nikhat Raghna

Repository: github.com/nikhattraghna/aqi-predictor (main)

Live App: <https://aqi-predictor-c3vmiz3nznbzrpnspitjaa.streamlit.app>

Domain: Time-Series Forecasting / Environmental ML

Target: PM_{2.5} ($\mu\text{g}/\text{m}^3$), Islamabad (33.6844°N, 73.0479°E)

Horizon: 72 hours (hourly resolution)

Models: Ridge, Random Forest, XGBoost, LightGBM

Champion: Ridge (smallest CV R^2 gap = 0.0356)

Feature Store: Hopsworks (Serverless, EU-West)

Serving: Streamlit (5 pages) + FastAPI (6 endpoints)

Orchestration: GitHub Actions (hourly + daily cron)

Executive Summary

The Islamabad AQI Forecasting System is a fully automated, production-grade machine-learning platform that forecasts hourly ground-level PM_{2.5} concentration for Islamabad, Pakistan, over a rolling 72-hour horizon. The system was engineered under a hard real-world constraint — only *three months* of historical hourly observations were obtainable — which forced every design decision to treat overfitting control and statistical honesty as first-class objectives.

This is not a notebook experiment; it is a closed-loop MLOps system. A **feature pipeline** (`hourly_feature_pipeline.py`) ingests fresh weather and air-quality observations from the keyless Open-Meteo API every hour and persists engineered features to a **Hopworks Feature Store** (`aqi_features, v1`). A **training pipeline** (`daily_training_pipeline.py`) runs daily at 00:30 UTC, training four model families (Ridge, Random Forest, XGBoost, LightGBM) under a leakage-safe 3-fold `TimeSeriesSplit` protocol with nested per-fold feature selection, evaluating each on a chronological holdout, and selecting a production champion by an overfitting-aware criterion. The champion — currently **Ridge** — its feature contract, and all dashboard artifacts are published to the Hopworks **Model Registry** and a **dataset bundle**. A **recursive multi-step forecaster** (`forecast_future.py`) produces a genuine forward forecast by combining Open-Meteo’s numerical weather forecast with autoregressive PM_{2.5} lags fed forward step by step. Continuous **drift monitoring** (Kolmogorov–Smirnov + Population Stability Index for data drift, AND-gated; baseline-relative RMSE tracking for model drift), **SHAP explainability**, and a five-page **Streamlit dashboard** close the loop. All orchestration is encoded in **GitHub Actions**; the system is stateless and reconstructs itself from Hopworks on every run.

Headline Engineering Result

On the chronological holdout (last 15%), all four models generalize well. The selected champion, **Ridge**, wins on *both* the selection criterion — the smallest cross-validated train–val R^2 gap (**0.0356**) — *and* raw holdout accuracy ($R^2 = 0.9788$, MAE $1.78 \mu\text{g}/\text{m}^3$), ahead of LightGBM ($R^2 = 0.9431$), Random Forest (0.9339), and XGBoost (0.9326). The system shows that with disciplined leakage control, an ℓ_2 -regularized linear model, and a gap-aware selection metric, a trustworthy forecaster can be built and operated continuously on just three months of data.

Abstract

Air pollution is a leading environmental health risk in South Asian megacities, yet operational, self-updating forecasting systems remain rare in the region. This report presents an end-to-end MLOps system for forecasting hourly $PM_{2.5}$ in Islamabad. The technical contributions are fourfold. First, a *leakage-safe* pipeline that performs feature selection *inside* cross-validation folds and constructs all rolling/lag features with an explicit one-step shift to prevent target leakage. Second, an *overfitting-aware model-selection protocol* that ranks candidates by the smallest cross-validated train–validation R^2 gap (tie-broken by highest validation R^2), explicitly favouring generalization over raw fit — a decision motivated by the small-data regime. Third, a *contract-driven inference architecture* (`feature_config.json`) that eliminates the class of train/serve feature-mismatch bugs by serializing the exact feature list, scaling requirement, and target alongside the model. Fourth, a *stateless, feature-store-centric deployment* in which every component — training CI, forecaster, and dashboard — reads from and writes to Hopsworks, removing the need to commit data or models to version control. We describe the recursive multi-step forecasting algorithm that produces true forward forecasts using numerical weather predictions as exogenous inputs, the AND-gated data/model drift detectors, the SHAP explainability layer, and the GitHub Actions automation that operates the system continuously. Empirical results on a chronological holdout confirm strong generalization (R^2 from 0.93 to 0.98, CV train–val gap ≤ 0.07 for all models).

Contents

Executive Summary	ii
Abstract	iii
Acronyms and Abbreviations	vii
Introduction	1
Context	1
What Was Built	1
The Defining Constraint	1
Problem Statement	1
Formal Definition	1
System Objectives	2
Functional Requirements	2
Non-Functional Requirements	3
Literature and Industry Background	3
Air-Quality Forecasting	3
MLOps Practice	3
Data Engineering Pipeline	4
Architecture Overview	4
Repository Structure	4
Data Sources	5
Ingestion (hourly_feature_pipeline.py)	5
Data Validation and Cleaning	5
Feature Engineering Pipeline	6
Feature Catalogue	6
Leakage Prevention (Shift-Before-Roll)	6
Feature Selection (Nested, Per-Fold)	6
Machine Learning Methodology	7
Time-Series Cross-Validation	7
Development / Holdout Split	7
Overfitting Metric and Verdicts	7
Model Development	8
Ridge Regression (Champion)	8
Random Forest	8
XGBoost	8
LightGBM	8
Hyperparameters (verbatim)	8
Model Evaluation	9
Metrics	9
Cross-Validated Results	9

Held-Out Test Results	10
Feature Importance (Random Forest Selector)	10
Model Selection Process	11
Selection Criterion	11
Promotion (<code>select_best_model.py</code>)	11
Selection Instability (Known Caveat)	11
Production Deployment	11
Stateless, Feature-Store-Centric Design	11
The Dashboard Bundle (<code>upload_dashboard_bundle.py</code>)	12
Serving: API and Dashboard	12
MLOps Architecture	12
The Two Scheduled Pipelines	12
The Feature Contract	13
Monitoring and Drift Detection	13
Data Drift (<code>data_drift.py</code>)	13
Model Drift (<code>model_drift.py</code>)	14
Alerts (<code>alerts.py</code>)	14
Explainability and Interpretability	14
SHAP (<code>shap_explainer.py</code>)	14
Graceful Degradation	14
Dashboard System	15
Architecture	15
Pages	15
Automation and CI/CD	15
GitHub Actions Workflows	15
Testing	16
Model Registry Integration	16
Forecasting Pipeline	16
True Forward Forecast (<code>forecast_future.py</code>)	16
Backtest (<code>forecast_next_3_days.py</code>)	17
Challenges and Solutions	17
Performance Analysis	18
Accuracy	18
Operational Performance	18
Cost	18
Limitations	19
Future Improvements	19
Lessons Learned	19
Conclusion	20
References	20

Unified Dependencies (<code>requirements.txt</code>)	21
Daily Training Pipeline Stage Order	21
AQI Category Mapping (<code>pm25_to_category</code>)	22
Per-Fold Cross-Validation Detail	22

List of Tables

4	Engineered feature families (candidate catalogue).	6
5	Cross-validated performance (3-fold TimeSeriesSplit), latest training run. CV train R^2 is shown as val + gap.	9
6	Final holdout performance (most recent $\approx 15\%$), latest training run.	10
7	Random Forest feature importances — the ranker used for nested feature selection (representative training run), not the champion model itself. The champion (Ridge) is explained via SHAP in the Explainability chapter.	10
8	Scheduled pipelines and their cron triggers (UTC).	12
9	Dashboard pages and content.	15
11	PM _{2.5} to AQI category bands $[lo, hi)$, verbatim from <code>forecast_future.py</code>	22
12	Per-fold CV metrics from a representative training run (illustrative; absolute values shift run to run). Ratio = val RMSE / train RMSE.	22

Acronyms and Abbreviations

Acronym	Definition
AQI	Air Quality Index
PM _{2.5}	Particulate Matter $\leq 2.5 \mu m$ in diameter
PM ₁₀	Particulate Matter $\leq 10 \mu m$ in diameter
MLOps	Machine Learning Operations
CI/CD	Continuous Integration / Continuous Deployment
CV	Cross-Validation
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
R^2	Coefficient of Determination
PSI	Population Stability Index
KS	Kolmogorov–Smirnov (test / statistic)
SHAP	SHapley Additive exPlanations
RF	Random Forest
XGBoost	eXtreme Gradient Boosting
LightGBM	Light Gradient Boosting Machine
NWP	Numerical Weather Prediction
FG	Feature Group (Hopsworks)
API	Application Programming Interface
UTC	Coordinated Universal Time
JSON	JavaScript Object Notation
ETL	Extract, Transform, Load

Introduction

Context

Islamabad, like much of the Indo-Gangetic plain, experiences hazardous particulate pollution driven by traffic, industry, residential combustion, regional dust transport, and seasonal meteorology. $PM_{2.5}$ — fine particulate matter small enough to penetrate deep into the lungs and bloodstream — is the pollutant most strongly associated with adverse health outcomes. A reliable short-horizon forecast enables individuals to plan outdoor activity, allows schools and employers to issue advisories, and gives authorities lead time for mitigation.

What Was Built

The repository `nikhatraghna/aqi-predictor` implements a continuously operating system that:

- ingests fresh weather and air-quality data hourly from Open-Meteo;
- maintains an engineered, point-in-time feature table in a Hopworks Feature Store;
- retrains and re-evaluates four model families daily;
- selects a production champion by smallest CV R^2 gap;
- produces a true 72-hour forward forecast plus a 72-hour backtest;
- monitors data and model drift and raises alerts;
- explains predictions with SHAP;
- serves a five-page Streamlit dashboard and a six-endpoint FastAPI service; and
- operates autonomously via GitHub Actions, with no local state.

The Defining Constraint

Only **three months** ($\approx 2,176$ hourly rows in a representative run) of history were obtainable. Small data makes overfitting the dominant failure mode, makes leakage catastrophic, and makes naive “best test score wins” selection statistically unsound. Every methodological choice in this report — nested feature selection, the R^2 -gap selection metric, very shallow trees, strong ℓ_2 regularization, and early stopping — is a direct response to this constraint.

Problem Statement

Formal Definition

Let $y_t \in \mathbb{R}_{\geq 0}$ denote the $PM_{2.5}$ concentration ($\mu g/m^3$) at hour t . Given pollutant and meteorological observations up to the present hour T , the task is to produce point forecasts

$$\hat{y}_{T+h}, \quad h = 1, 2, \dots, 72,$$

for the next 72 hours. Inference has access to (i) the autoregressive history of y , (ii) co-located pollutant series (PM_{10} , NO_2 , SO_2 , CO , dust), and (iii) a *numerical weather forecast* of meteorological drivers for the forecast window from Open-Meteo.

System Objectives

1. Forecast hourly $PM_{2.5}$ for Islamabad over a 72-hour forward horizon with honest, generalizing accuracy.
2. Eliminate overfitting through leakage-safe validation and an overfitting-aware selection metric.
3. Operate continuously and autonomously (hourly ingestion, daily retraining) with no manual steps and no local state.
4. Guarantee train/serve consistency via a serialized feature contract.
5. Detect and surface data and model drift; recommend retraining.
6. Explain predictions with SHAP for trust and debuggability.
7. Serve a public, always-on dashboard independent of any notebook session.
8. Keep secrets out of version control; reproduce from the feature store.

Functional Requirements

ID	Requirement
FR-1	Ingest weather and air-quality observations from Open-Meteo hourly.
FR-2	Engineer lag, rolling, calendar, and exogenous features; persist to feature group <code>aqi_features</code> (v1).
FR-3	Train Ridge, RF, XGBoost, LightGBM daily under 3-fold TimeSeriesSplit CV with nested per-fold feature selection.
FR-4	Write one metrics JSON per model to <code>models/metrics/<model>.json</code> plus a consolidated <code>model_metrics.json</code> .
FR-5	Select the champion by smallest CV R^2 gap, tie-broken by highest CV validation R^2 .
FR-6	Promote champion + <code>feature_config.json</code> ; publish to Hopworks registry and dataset bundle.
FR-7	Produce a true 72-hour forward forecast (<code>forecast_3days.parquet</code>).
FR-8	Produce a 72-hour backtest (<code>hindcast_3days.parquet</code>).
FR-9	Compute data drift (KS + PSI, AND-gated) and model drift (baseline-relative RMSE); emit alerts.
FR-10	Compute SHAP feature importances for the champion; persist them.
FR-11	Serve a five-page Streamlit dashboard reading exclusively from the Hopworks bundle, plus a FastAPI service.
FR-12	Orchestrate all of the above via GitHub Actions cron workflows.

Non-Functional Requirements

Attribute	Requirement / Realization
Reliability	API calls use a retry session (<code>Retry(total=4, backoff_factor=2, status_forcelist=[429,500,502,503,504])</code> , <code>timeout=30s</code>).
Reproducibility	<code>random_state=42</code> ; deterministic 3-fold <code>TimeSeriesSplit</code> ; artifacts versioned in the registry.
Statelessness	<code>ensure_features_exist()</code> restores the feature table from Hopworks when local parquet is absent.
Maintainability	Single unified <code>requirements.txt</code> ; modular <code>src/</code> packages; contract-driven inference.
Security	Secrets only in <code>.env</code> (gitignored) and GitHub Actions secrets; README uses placeholders.
Portability	Dockerized (<code>python:3.11-slim</code>); Streamlit Cloud pinned to Python 3.11; <code>packages.txt</code> declares <code>build-essential</code> .
Observability	Per-model metrics, drift reports, alerts, and drift history persisted and surfaced on the dashboard.
Availability	Dashboard hosted on Streamlit Cloud, independent of any local/Colab session.

Literature and Industry Background

Air-Quality Forecasting

Approaches range from ARIMA and exponential smoothing to gradient-boosted trees and deep sequence models (LSTM, Temporal Fusion Transformers). For short horizons with strong autocorrelation and limited data, gradient-boosted trees and regularized linear models are well established as strong, sample-efficient baselines that resist overfitting better than high-capacity neural networks — the reason this project standardizes on Ridge, RF, XGBoost, and LightGBM. (Experimental `train_lstm.py`, `train_prophet.py`, and `train_lasso.py` modules exist in the repository but are deliberately excluded from the production four-model lineup.)

MLOps Practice

The system embodies mainstream MLOps patterns: a *feature store* (Hopworks) for point-in-time features and train/serve parity; a *model registry* for versioned, promotable artifacts; *CI/CD* (GitHub Actions) for scheduled pipelines; and *continuous monitoring* for drift — mirroring the reference architectures of Google’s MLOps maturity model and the feature-store literature.

Data Engineering Pipeline

Architecture Overview

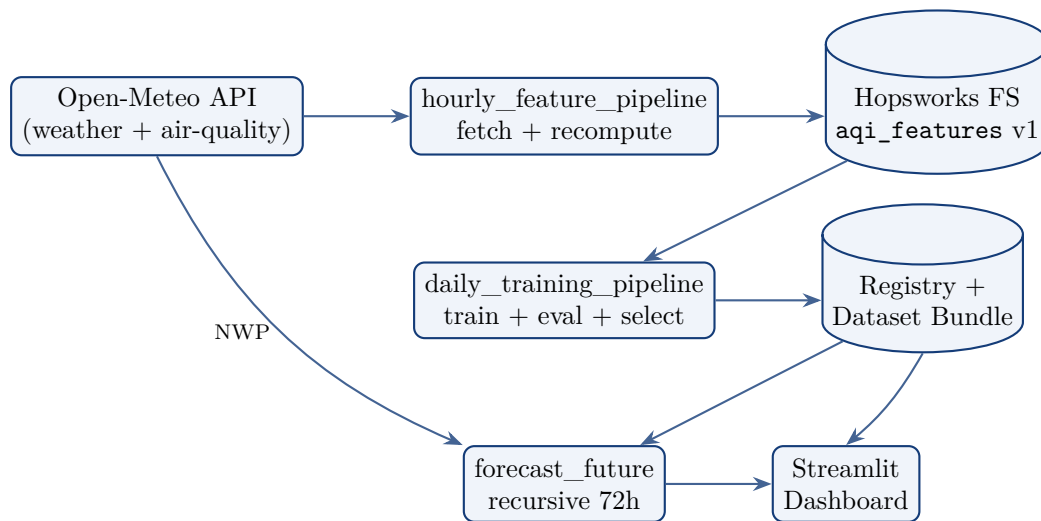


Figure 1: High-level data and control flow. The Feature Store and Registry are the integration backbone; every component is stateless and reconstructs from them.

Repository Structure

```

1 aqi-predictor/
2   src/
3     automation/ hourly_feature_pipeline.py daily_training_pipeline.py
4                 upload_dashboard_bundle.py
5   data_pipeline/ fetch_realtime_data.py fetch_historical_air_quality.py
6                 preprocess.py validate_data.py
7   feature_engineering/ create_features.py feature_selector.py
8   config/ features.py
9   training/ train_ridge.py train_random_forest.py train_xgboost.py
10            train_lightgbm.py evaluate_models.py select_best_model.py
11   inference/ forecast_future.py forecast_next_3_days.py
12            load_model.py predict.py
13   monitoring/ data_drift.py model_drift.py alerts.py
14   explainability/ shap_explainer.py lime_explainer.py
15   models/ upload_model_to_registry.py download_best_model.py ...
16   feature_store/ hopsworks_connection.py upload_features.py
17                 fetch_features.py
18   api/ fastapi_app.py
19   dashboard/ Home.py utils.py charts.py alerts.py
20              pages/01_Forecast.py 02_Explanations.py 03_EDA.py
21              04_Monitoring.py
22   models/ metrics/<model>.json model_metrics.json best_model.json
23           best_model_metrics.json best_model/{model.pkl,
24           feature_config.json}
25   reports/ drift/*.json drift/model_drift_history.parquet
26           shap/shap_summary.png
27   .github/workflows/ feature_pipeline.yml training_pipeline.yml
28           tests.yml docker.yml
29   tests/ Dockerfile requirements.txt packages.txt README.md

```

Listing 1: Project layout (root-level packages and key artifacts).

Data Sources

The **model** is standardized on a single source — the keyless **Open-Meteo** API — to guarantee train/serve parity:

- **Weather forecast:** <https://api.open-meteo.com/v1/forecast> (temperature and related drivers).
- **Air quality:** <https://air-quality-api.open-meteo.com/v1/air-quality> (PM_{2.5}, PM₁₀, NO₂, SO₂, CO, O₃, dust).

Both are queried for Islamabad (lat 33.6844, lon 73.0479) with `timezone="Asia/Karachi"`. AQICN and OpenWeather are referenced in the README tech stack but are *display-only*; mixing sources between training and serving was identified as a train/serve-skew hazard and kept off the model path.

Engineering Decision: One Source for the Model

Forecast accuracy is dominated by consistency, not by the number of sources. Standardizing the model exclusively on Open-Meteo ensures exogenous inputs at inference are produced identically to training. Auxiliary sources are used only for human display.

Ingestion (hourly_feature_pipeline.py)

The hourly job: `ensure_features_exist()` restores the table from Hopsworks if the local parquet is missing (a fresh CI runner); `fetch_open_meteo()` pulls the latest observations (`past_days=1`, `forecast_days=1`) through a retry session and localizes timestamps to UTC; `recompute_features()` rebuilds lag/rolling/time features on the extended frame; `build_new_base_row()` and `upload_new_row()` append the new row(s) to the feature group.

```
1 from requests.adapters import HTTPAdapter
2 from urllib3.util.retry import Retry
3
4 def _session():
5     s = requests.Session()
6     retry = Retry(total=4, backoff_factor=2,
7                 status_forcelist=[429, 500, 502, 503, 504],
8                 allowed_methods=["GET"])
9     s.mount("https://", HTTPAdapter(max_retries=retry))
10    return s
```

Listing 2: Resilient HTTP session shared by ingestion and the forecaster.

Data Validation and Cleaning

The `data_pipeline/` package (`validate_data.py`, `preprocess.py`) enforces schema and cleaning; the `recompute` step normalizes timestamps to a single **tz-aware UTC** convention (Open-Meteo’s Karachi-local timestamps are localized to match the stored convention), de-duplicates hours, and preserves chronological order — a prerequisite for correct lag computation and valid time-series CV.

Engineering Decision: Timezone Discipline

A subtle high-impact bug class is silent timezone mismatch. The pipeline pins a single UTC convention and localizes every external timestamp into it, so lags and rolling windows align across ingestion, training, and forecasting.

Feature Engineering Pipeline

Feature Catalogue

`src/config/features.py` defines `FEATURE_COLUMNS` (34 candidate columns) and `TARGET_COLUMN = "pm25"`:

Table 4: Engineered feature families (candidate catalogue).

Family	Members
Co-pollutants / exo	pm10, co, no2, so2, o3, aod, dust
Meteorology	temperature, humidity, precipitation, pressure, wind_speed, wind_direction, cloud_cover, weather_code
Autoregressive lags	pm25_lag_{1,3,6,12,24}
Rolling mean	pm25_rol1_mean_{6,12,24} (built); pm25_rol1_mean_3 (in contract)
Rolling std.	pm25_rol1_std_{3,6,12} (in contract)
Calendar	hour, day_of_week, month, day, hour_sin, hour_cos

Leakage Prevention (Shift-Before-Roll)

In `create_features.py` every rolling feature is computed on a *shifted* target so the feature for hour t uses only information strictly before t :

$$\text{pm25_rol1_mean_k}(t) = \frac{1}{k} \sum_{i=1}^k y_{t-i}, \quad \text{pm25_lag_j}(t) = y_{t-j}.$$

Concretely: `df["pm25"].shift(1).rolling(window=k).mean()`. The lags built are $\{1, 3, 6, 12, 24\}$ and rolling means $\{6, 12, 24\}$; a code comment records that `rol1_mean_3` and the `rol1_std` variants were removed from the batch builder to curb leakage/overfitting. Calendar features use a cyclical encoding `hour_sin = sin(2 $\pi$  hour/24)`, `hour_cos = cos(2 $\pi$  hour/24)`.

Engineering Decision: Shift-Before-Roll

Computing a rolling mean that includes the current observation is a classic leakage bug that inflates offline scores and collapses in production. The pipeline shifts the target by one hour *before* any rolling operation, so every feature is strictly causal.

Feature Selection (Nested, Per-Fold)

Selection runs *inside each CV fold, on the training portion only* (`feature_selector.py`):

- **Ridge:** `SelectKBest(f_regression, k=12)`, then force-append must-haves `pm25_lag_1`, `pm25_lag_24`, `pm25_rol1_mean_24` (yielding the 13 features in `ridge.json`).
- **Tree models (RF/XGBoost):** Random Forest importance, retaining the top features (top-10 in the production runs).

The champion (Ridge) uses a 13-feature selection (`requires_scaling=true`); a representative tree-model selection (top-10 by RF importance) is:

```
pm25_rol1_mean_3, pm25_lag_1, pm10, no2, pm25_rol1_std_3, so2, pm25_rol1_std_6,
pm25_lag_3, co, dust
```

Engineering Decision: Nested Selection

Selecting features on the full dataset before CV leaks validation information into the selection step. Nesting selection inside each fold keeps every validation block genuinely unseen — the single most important guard against optimistic CV scores in small data.

Machine Learning Methodology

Time-Series Cross-Validation

Because the data are temporally ordered, random k -fold CV would train on the future to predict the past. All four trainers use `TimeSeriesSplit` with `N_SPLITS=3`: each fold trains on a contiguous prefix and validates on the subsequent block, so training always precedes validation in time.

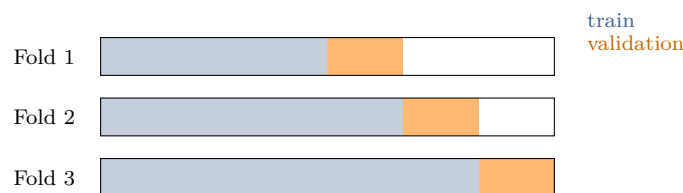


Figure 2: Expanding-window `TimeSeriesSplit` (3 folds); feature selection is re-run on each train block.

Development / Holdout Split

The series is split chronologically: a holdout test block of `TEST_SIZE = max(⌊0.15  $n$ ⌋, 24)` (the most recent $\approx 15\%$; e.g. 326 of 2,176 rows in a representative run) is held out and touched once; the remainder is used for CV and the final fit.

Overfitting Metric and Verdicts

The headline diagnostic is the **train–validation R^2 gap** $\Delta R^2 = R_{\text{train}}^2 - R_{\text{val}}^2$, classified by `classify_gap()`:

$\Delta R^2 \leq 0.03 \Rightarrow$ EXCELLENT, $\leq 0.05 \Rightarrow$ VERY GOOD, $\leq 0.10 \Rightarrow$ ACCEPTABLE, else $\Rightarrow$ OVERFIT

The RMSE ratio ($\text{RMSE}_{\text{val}}/\text{RMSE}_{\text{train}}$, median across folds) is recorded only as *informational*: it is misleading for high-variance models because a tiny training RMSE inflates the ratio even when validation error is excellent.

Engineering Decision: R^2 -Gap over RMSE-Ratio

XGBoost’s median fold RMSE ratio is $1.82\times$ (would read as “concerning”), driven purely by a tiny train RMSE; yet its CV val R^2 is 0.954 and its CV gap only 0.040. The gap tells the truth: the model generalizes. The project therefore adopts the R^2 gap as both the overfitting metric and the selection criterion.

Model Development

Four families are trained, each in its own module, each writing its own metrics JSON. Hyperparameters are intentionally conservative for the small-data regime.

Ridge Regression (Champion)

Ridge minimizes least squares with an ℓ_2 penalty:

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad \hat{\beta} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$

`train_ridge.py` tunes λ (alpha) by inner `GridSearchCV` over $\{0.01, 0.1, 1, 10, 50, 100, 500\}$ with a nested `TimeSeriesSplit`; the selected $\alpha = 0.01$. Ridge is the only model requiring `StandardScaler` (`requires_scaling=true`). Being linear, it has an inherently small train–val gap — the smallest of the four in the latest run (0.0356), which is why it is the selected champion.

Random Forest

An RF averages B de-correlated trees on bootstrap samples: $\hat{y}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x})$. Bagging reduces variance; depth/leaf limits control overfitting. RF also serves as the importance ranker for nested tree-model selection.

XGBoost

XGBoost is a second-order gradient-boosted ensemble minimizing

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|_2^2,$$

with optimal leaf weight and structure score (gradients g_i , Hessians h_i):

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \text{score} = -\frac{1}{2} \sum_j \frac{\left(\sum_{i \in I_j} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T.$$

Early stopping (patience 50) halts training at the best iteration (observed `best_iteration=3964` of a 4000 ceiling), preventing over-growth. Note the extreme regularization: `max_depth=2`, `reg_lambda=15`, `gamma=1`, `min_child_weight=20`.

LightGBM

LightGBM grows trees *leaf-wise* (best-first) with histogram binning and Gradient-based One-Side Sampling. It optimizes the same regularized boosting objective as XGBoost, regularized here by `num_leaves=7`, `max_depth=3`, `min_child_samples=20`, and early stopping.

Hyperparameters (verbatim)

```
1 # train_ridge.py
2 ALPHA_GRID = {"alpha": [0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 500.0]} # best =
   0.01
3 scaler = StandardScaler() # requires_scaling = True ; N_SPLITS = 3
4
5 # train_random_forest.py
```

```

6 RF_PARAMS = dict(n_estimators=400, max_depth=4, min_samples_split=40,
7                  min_samples_leaf=20, max_features=0.4, max_samples=0.7,
8                  random_state=42, n_jobs=-1) # no scaling ; no early
                                                stopping
9
10 # train_xgboost.py
11 XGB_PARAMS = dict(n_estimators=4000, max_depth=2, learning_rate=0.02,
12                  subsample=0.6, colsample_bytree=0.5, min_child_weight=20,
13                  reg_alpha=3.0, reg_lambda=15.0, gamma=1.0,
14                  early_stopping_rounds=50, objective="reg:squarederror",
15                  random_state=42) # best_iteration = 3964
16
17 # train_lightgbm.py
18 LGB_PARAMS = dict(n_estimators=4000, learning_rate=0.02, max_depth=3,
19                  num_leaves=7, min_child_samples=20, subsample=0.6,
20                  subsample_freq=1, colsample_bytree=0.5,
21                  reg_alpha=3.0, reg_lambda=15.0) # early_stopping(50)

```

Listing 3: Production hyperparameters per model (from the training modules).

Engineering Decision: Aggressive Regularization

Tree depth is capped at 2 (XGBoost) and 3 / 7 leaves (LightGBM), with large minimum child weights/samples and heavy ℓ_2 . On three months of data, shallow, strongly-regularized boosters are the difference between a model that generalizes and one that memorizes.

Model Evaluation

Metrics

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Cross-Validated Results

Table 5: Cross-validated performance (3-fold TimeSeriesSplit), latest training run. CV train R^2 is shown as val + gap.

Model	CV Train R^2	CV Val R^2	ΔR^2 (gap)	Verdict
Ridge (champion)	0.9777	0.9421	0.0356	VERY GOOD
XGBoost	0.9937	0.9539	0.0398	VERY GOOD
LightGBM	0.9947	0.9532	0.0415	VERY GOOD
Random Forest	0.9542	0.8865	0.0677	ACCEPTABLE

Held-Out Test Results

Table 6: Final holdout performance (most recent $\approx 15\%$), latest training run.

Model	Test MAE	Test RMSE	Test R^2	Train–Test gap
Ridge (champion)	1.7815	2.5813	0.9788	0.0035
LightGBM	2.5811	4.2329	0.9431	0.0531
Random Forest	3.0568	4.5620	0.9339	0.0210
XGBoost	2.8359	4.6071	0.9326	0.0605

Notable Result: The Champion Wins on Both Criteria

In the latest run, **Ridge** simultaneously attains the smallest CV gap (**0.0356**) and the best holdout accuracy ($R^2 = 0.9788$, MAE 1.78), so the generalization-first selection metric and raw accuracy agree. This is not guaranteed: the top three models are near-tied on CV gap (Ridge 0.0356, XGBoost 0.0398, LightGBM 0.0415), so the champion can flip between runs as the data window grows — a documented instability (discussed under *Model Selection Process* and *Limitations*). As a regularized linear model, Ridge’s near-zero train–test gap (0.0035) reflects genuinely low variance.

(placeholder)

Figure 3: Per-model test R^2 and RMSE comparison.

Feature Importance (Random Forest Selector)

Table 7: Random Forest feature importances — the ranker used for nested feature selection (representative training run), not the champion model itself. The champion (Ridge) is explained via SHAP in the Explainability chapter.

Feature	Importance
pm25_roll_mean_3	0.4139
pm25_lag_1	0.3078
pm10	0.1186
no2	0.0863
pm25_roll_std_6	0.0286
pm25_lag_3	0.0143
co	0.0103
so2	0.0103
pm25_roll_std_3	0.0077
dust	0.0023

pm25_roll_mean_3 and pm25_lag_1 together carry $\approx 72\%$ of importance, confirming the strong short-range autocorrelation of $PM_{2.5}$, with PM_{10} and NO_2 as the leading exogenous signal.

Model Selection Process

Selection Criterion

`evaluate_models.py` consumes the per-model JSON, drops candidates missing CV metrics (this is why `prophet` — which has no `cv` block — is excluded), and sorts:

```
1 SORT_COLS = ["CV_Gap", "CV_Val_R2"]
2 SORT_ASC  = [True,      False]          # smallest gap; tie-break highest val
                                           R2
3 df = df.dropna(subset=["CV_Gap", "CV_Val_R2"])
4 best = df.sort_values(SORT_COLS, ascending=SORT_ASC).iloc[0]
5 # writes best_model.json = {"best_model": <name>}
6 #      best_model_metrics.json (selected_by = "min_cv_gap_then_cv_val_r2")
```

Listing 4: Selection logic in `evaluate_models.py` (overfitting-first, accuracy tie-break).

Applied to the latest run, the CV gaps are Ridge 0.0356 < XGBoost 0.0398 < LightGBM 0.0415 < RF 0.0677, so **Ridge** is selected — written to `best_model.json = {"best_model": "ridge"}` and reflected as the dashboard's displayed champion.

Promotion (`select_best_model.py`)

A promotion utility reads `best_model.json`, copies the artifact to `models/best_model/`, copies the scaler only if the model needs it (Ridge only), and writes the inference contract. (Minor labelling note: the contract's `selected_by` string is the hard-coded `"cv_val_r2"` while `best_model_metrics.json` records the true criterion `"min_cv_gap_then_cv_val_r2"`; the operative logic is the latter.)

Selection Instability (Known Caveat)

Because the criterion favours the smallest gap, and Ridge, XGBoost, and LightGBM sit within ~ 0.006 of one another on CV gap, a different data window seen by two environments (e.g. Colab vs. the GitHub runner, which fetch from Open-Meteo at slightly different times) can flip the winner. The operational resolution is governance, not code: designate **GitHub Actions as the single production uploader**, so the registry and dashboard always reflect one reproducible run. The forecast's provenance tag (`local` vs. a numeric registry version) makes this visible at a glance.

Engineering Decision: One Production Uploader

Both Colab and CI could publish to Hopworks (last-writer-wins), so the displayed champion could flip. The fix is governance: only scheduled CI uploads production artifacts; local runs are for experimentation. The dashboard then always shows a versioned registry artifact, never a `local` one.

Production Deployment

Stateless, Feature-Store-Centric Design

No data or model binaries drive serving from git. The **feature group** `aqi_features` (v1) holds features; the **model registry** holds versioned champions; the **dataset API** holds a single `dashboard_bundle.zip` under `Resources/aqi_dashboard/`. Any runner reconstructs what it needs from Hopworks at start-up.

The Dashboard Bundle (`upload_dashboard_bundle.py`)

The uploader zips exactly the artifacts the dashboard needs (existence-guarded) and uploads with `overwrite=True`:

```

1 models/model_metrics.json           reports/drift/data_drift_report.json
2 models/best_model.json              reports/drift/model_drift_report.json
3 models/best_model_metrics.json      reports/drift/alerts_report.json
4 data/processed/forecast_3days.parquet
   reports/drift/model_drift_history.parquet
5 data/processed/islamabad_features.parquet
   reports/shap/shap_importance.parquet
6 data/processed/current_conditions.json
   data/processed/hindcast_3days.parquet
7 + models/best_model/** (model.pkl, feature_config.json, scaler if present)

```

Listing 5: Bundle FILES list plus the recursive `models/best_model/` contents.

Serving: API and Dashboard

A **FastAPI** service (`src/api/fastapi_app.py`, six endpoints) is containerized via the `Dockerfile` (`python:3.11-slim + build-essential, libgomp1; pip install -r requirements.txt fastapi uvicorn; EXPOSE 8000; CMD uvicorn src.api.fastapi_app:app`). The **Streamlit** dashboard is hosted on Streamlit Cloud, pinned to Python 3.11 (avoiding a Python 3.14/protobuf meta-class incompatibility), with `packages.txt` declaring `build-essential` so native dependencies (the Hopworks → pyjks → twofish chain) compile. The dashboard is publicly accessible at <https://aqi-predictor-c3vmiz3nznbzrpnspitjaa.streamlit.app>, running 24/7 independently of any developer or notebook session.

Engineering Decision: Single `requirements.txt`

One unified dependency manifest serves training, inference, dashboard, and API, trading a slightly larger install footprint for guaranteed dependency parity across every runner. The `Dockerfile` layers `fastapi/uvicorn` on top for the API container.

MLOps Architecture

The Two Scheduled Pipelines

Table 8: Scheduled pipelines and their cron triggers (UTC).

Pipeline	Cron	Responsibilities
Feature (hourly)	0 * * * *	Ingest Open-Meteo, recompute features, append to feature group.
Training (daily)	30 0 * * *	Refresh features, then train 4 models → evaluate → promote → forecast → backtest → drift → alerts → SHAP → registry → bundle.

`daily_training_pipeline.py` runs each stage as `subprocess.run([sys.executable, "-m", module])`. The four trainers, `evaluate_models`, and `select_best_model` are *critical* (failure raises and aborts); forecasting, drift, alerts, SHAP, and the two uploads are *best-effort* (failures are collected and logged but do not lose the run). See the *Daily Training Pipeline Stage Order* appendix for the exact 14-stage order.

(placeholder)

Figure 4: End-to-end MLOps workflow (sequence diagram).

The Feature Contract

The cornerstone of train/serve correctness is `models/best_model/feature_config.json` (verbatim committed contents):

```

1 {
2   "model_name": "ridge",
3   "target": "pm25",
4   "requires_scaling": true,
5   "features":
6     ["pm10", "co", "no2", "o3", "pm25_lag_1", "pm25_lag_3", "pm25_lag_24",
7      "pm25_roll_mean_3", "pm25_roll_std_3", "pm25_roll_mean_6",
8      "pm25_roll_std_6", "pm25_roll_std_12", "pm25_roll_mean_24"],
9   "n_features": 13,
10  "selected_by": "cv_val_r2",
11  "cv_val_r2": 0.9421,
12  "cv_r2_gap": 0.0356,
13  "test_metrics": {"mae": 1.7815, "rmse": 2.5813, "r2": 0.9788},
14  "test_r2_gap": 0.0035
15 }
```

Listing 6: `feature_config.json` — the inference contract (committed).

Engineering Decision: Contract-Driven Inference

Serializing the exact feature order, scaling flag, and target with the model eliminates the entire class of train/serve feature-mismatch bugs. Inference code never hard-codes a feature list; it reads the contract and constructs the design matrix to match the model that was actually selected.

Monitoring and Drift Detection

Data Drift (`data_drift.py`)

Two complementary tests compare a recent 72-observation window against a training baseline, per feature.

Kolmogorov–Smirnov (via `scipy.stats.ks_2samp`) — the maximum gap between empirical CDFs, with a significance cutoff $p < 0.05$:

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{cur}}(x)|.$$

Population Stability Index — 10 reference-quantile bins with proportions e_i (expected) and a_i (actual), $\epsilon = 10^{-6}$ smoothing:

$$\text{PSI} = \sum_{i=1}^{10} (a_i - e_i) \ln\left(\frac{a_i}{e_i}\right),$$

thresholds `PSI_WARN= 0.10`, `PSI_DRIFT= 0.20`.

```

1 def compute_psi(expected, actual, bins=10, eps=1e-6):
2     edges = np.quantile(expected, np.linspace(0, 1, bins + 1))
3     edges[0], edges[-1] = -np.inf, np.inf
4     e = np.histogram(expected, edges)[0] / len(expected)
```

```

5     a = np.histogram(actual, edges)[0] / len(actual)
6     e, a = np.clip(e, eps, None), np.clip(a, eps, None)
7     return float(np.sum((a - e) * np.log(a / e)))

```

Listing 7: PSI implementation (binned, ϵ -guarded).**Engineering Decision: AND-Gated Drift**

A feature is flagged DRIFT only when *both* the KS test is significant ($p < 0.05$) *and* $\text{PSI} \geq 0.20$; either alone yields at most WARNING. Requiring consensus from two tests of differing sensitivity suppresses noisy single-test alerts that would erode trust in the monitor.

Model Drift (model_drift.py)

Model drift is tracked relative to the baseline RMSE drawn from the contract's `test_metrics.rmse`. Degradation = $(\text{RMSE}_{\text{current}} - \text{RMSE}_{\text{baseline}}) / \text{RMSE}_{\text{baseline}}$ over the recent 72-hour window: $\geq \text{WARN_PCT} = 0.20$ raises **WARNING**; $\geq \text{DRIFT_PCT} = 0.40$ raises **DRIFT**. A rolling `model_drift_history.parquet` preserves the trend (timestamp, status, degradation %, current/baseline RMSE, worst PSI).

Alerts (alerts.py)

Severity is mapped NORMAL=0, WARNING=1, DRIFT=2; the overall state is `max(data, model)`. Data severity is escalated by the fraction of features flagged (`DATA_WARN_FRAC=0.20`, `DATA_DRIFT_FRAC=0.50`). The retraining recommendation is driven by *model* drift only: `retrain_recommended = (model.severity == "DRIFT")`. Output is `reports/drift/alerts_report.json`; delivery is console-only (Slack/email left as a TODO).

(placeholder)

Figure 5: Monitoring dashboard page: drift status and health.

Explainability and Interpretability

SHAP (shap_explainer.py)

The module is contract-driven: it loads the champion and its `feature_config.json`, reads the 200 most-recent rows of `islamabad_features.parquet`, builds `shap.Explainer(model, X)` (auto-selecting the right explainer for the champion), and persists global importances (mean|SHAP| per feature) to `reports/shap/shap_importance.parquet` (and `.csv`), with an optional `shap_summary.png`. SHAP attributes each prediction via the Shapley value:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)], \quad f(\mathbf{x}) = \phi_0 + \sum_j \phi_j.$$

Graceful Degradation

The Explanations page prefers the SHAP artifact when present in the bundle and falls back to the model's native feature importance otherwise — so the page is always informative even if the (best-effort) SHAP stage was skipped on a run.

(placeholder)

Figure 6: SHAP global feature-importance summary.

Dashboard System

Architecture

The Streamlit app is a `Home.py` plus four pages. `utils.py` owns a cached `_bootstrap_from_hopsworks()` (`@st.cache_resource`) that downloads and unzips the bundle once per session and sets `PROJECT_ROOT` to the bundle directory *before* any path is derived; every loader (`load_forecast`, `load_features`, `load_model_metrics`, `load_best_metrics`, `load_contract`, `load_alerts`, `load_model_drift`, `load_data_drift`, `load_hindcast`, `load_shap_importance`, ...) reads from that location. `charts.py` and `alerts.py` provide reusable Altair charts and alert cards.

Engineering Decision: Read Everything From Hopsworks
The dashboard never reads local repo files — only the downloaded bundle. This decouples the public app from any developer machine or notebook session and guarantees it shows exactly the artifacts CI published.

Pages

Table 9: Dashboard pages and content.

Page	Content
Home	Current conditions, champion summary, headline KPIs.
01_Forecast	72-hour forward forecast (chart + AQI categories) and backtest.
02_Explanations	SHAP / native importances; contract and selection rationale.
03_EDA	Historical series, distributions, correlation heatmap.
04_Monitoring	Data/model drift, health state, drift-history trend.

(placeholder)

Figure 7: Production dashboard — Forecast page.

Automation and CI/CD

GitHub Actions Workflows

Four workflows under `.github/workflows/`: `feature_pipeline.yml` (hourly), `training_pipeline.yml` (daily), `tests.yml` (push/PR on main), and `docker.yml` (image build). Each pipeline injects `HOPSWORKS_API_KEY` and `HOPSWORKS_PROJECT` from repository secrets, uses Python 3.11, installs the unified `requirements.txt`, and runs its module. Because the system is stateless, no git commit-back is needed — all outputs go to Hopsworks.

```
1 on:
2   schedule:
```

```

3   - cron: "0 * * * *"           # hourly, on the hour (UTC; best-effort
    timing)
4   workflow_dispatch: {}
5   jobs:
6     ingest:
7       runs-on: ubuntu-latest
8       env:
9         HOPSWORKS_API_KEY: ${ secrets.HOPSWORKS_API_KEY }
10        HOPSWORKS_PROJECT: ${ secrets.HOPSWORKS_PROJECT }
11      steps:
12        - uses: actions/checkout@v4
13        - uses: actions/setup-python@v5
14          with: { python-version: "3.11" }
15        - run: pip install -r requirements.txt
16        - run: python -m src.automation.hourly_feature_pipeline

```

Listing 8: Feature pipeline workflow (abridged, verbatim cron).

The daily workflow (cron: "30 0 * * *", concurrency group `training-pipeline`, `cancel-in-progress: false`) first runs `hourly_feature_pipeline` to refresh features, then `daily_training_pipeline`.

Engineering Note: Cron Timing

GitHub Actions cron is best-effort and can be delayed under load; the system tolerates this because ingestion is idempotent (de-duplicated by hour) and the dashboard always reads the latest published bundle.

Testing

`tests/` (with a `conftest.py` adding the repo root to `sys.path`) covers AQI categorization (`pm25_to_category` from `forecast_future.py`), PSI computation, artifact existence/shape, and basic inference — a fast correctness gate run on every push/PR via `python -m pytest -q`.

Model Registry Integration

`src/models/upload_model_to_registry.py` publishes the champion to the Hopsworks Model Registry with its metrics and contract; companion modules (`download_best_model.py`, `download_model_from_` restore it on a fresh runner. The forecaster prefers the registry artifact (tagging the forecast with the registry *version number*); when it loads a local file it tags the version as `local` — a deliberate provenance signal. A forecast showing `version = local` therefore did not originate from a CI/registry promotion, which is exactly the signal used to diagnose the champion-flip discussed under *Model Selection Process*.

Engineering Decision: Provenance via Version Tag

Tagging forecasts with `local` vs. a numeric registry version turns an invisible operational question (which environment produced this?) into a one-glance, debuggable field on the dashboard.

Forecasting Pipeline

True Forward Forecast (`forecast_future.py`)

The system produces a genuine *forward* forecast (`HORIZON=72`), not a hindcast. For each future hour $h = 1 \dots 72$:

1. Exogenous drivers (temperature and the contract's pollutants) come from the **Open-Meteo numerical forecast** (`forecast_days=4` on both the weather and air-quality endpoints).
2. Autoregressive features (`pm25_lag_{1,3,6,12,24}`), rolling mean/std over windows $\{3, 6, 12, 24\}$ are rebuilt from the *growing* sequence of *predicted* PM_{2.5}, fed forward recursively.
3. The champion (loaded via the contract; scaled iff `requires_scaling`) predicts $\hat{y}_{T+h}$, which is appended and used to build the next step's lags.

Output: `data/processed/forecast_3days.parquet`, with an AQI category per hour via `pm25_to_category` (see the AQI-category mapping appendix).

```

1 pm = list(recent_pm25)                                # most recent observed PM2.5
2 rows = []
3 for ts in future_index:                                # next 72 hourly timestamps
4     feat = {f"pm25_lag_{j}": pm[-j] for j in (1, 3, 6, 12, 24)}
5     for w in (3, 6, 12, 24):
6         feat[f"pm25_roll_mean_{w}"] = float(np.mean(pm[-w:]))
7         feat[f"pm25_roll_std_{w}"] = float(np.std(pm[-w:]))
8     feat.update(nwp_exogenous_at(ts)) # temperature, pm10, no2, ... from
        Open-Meteo
9     X = build_design_matrix(feat, contract["features"])
10    if contract["requires_scaling"]:
11        X = scaler.transform(X)
12    yhat = float(model.predict(X)[0])
13    pm.append(yhat) # feed prediction forward
14    label, emoji = pm25_to_category(yhat)
15    rows.append({"datetime": ts, "pm25": yhat, "aqi": label})

```

Listing 9: Recursive multi-step forecasting (core loop, abridged).

Engineering Decision: Forward Forecast over Hindcast

The core product requirement was a forecast of the *future*. Using Open-Meteo's NWP for exogenous drivers and recursively feeding predicted PM_{2.5} lags forward yields true future-dated predictions, while a separate backtest preserves an honest accuracy view against recent actuals.

Backtest (`forecast_next_3_days.py`)

A complementary backtest writes `hindcast_3days.parquet` with columns `datetime`, `actual_pm25`, `predicted_pm25` over the most recent 72 observed hours, letting the dashboard show recent predictive accuracy alongside the forward forecast.

(placeholder)

Figure 8: Forward 72-hour forecast and recent backtest overlay.

Challenges and Solutions

Challenge	Solution
Overfitting on 3-month data	Leakage-safe nested CV, shift-before-roll features, very shallow trees (<code>max_depth=2/num_leaves=7</code>), heavy ℓ_2 , early stopping, R^2 -gap selection.

Challenge	Solution
Misleading RMSE ratio	Adopted train–val R^2 gap as the overfitting metric; RMSE ratio kept informational.
Train/serve feature skew	Serialized <code>feature_config.json</code> contract consumed by all inference code.
Train/serve data skew	Standardized the model exclusively on Open-Meteo; demoted other APIs to display-only.
API timeouts (empty forecast)	Retry session (<code>Retry total=4, backoff=2</code>) + 30s timeout in ingestion and forecaster.
<code>NameError: Retry</code>	Added the missing <code>HTTPAdapter/Retry</code> imports to the ingestion module.
Champion flips (Ridge vs. XGBoost)	Near-tied CV gaps; resolved operationally by making CI the single production uploader.
Stateless CI	<code>ensure_features_exist()</code> restores the feature table from Hopsworks when local parquet is absent.
Streamlit blank / getpass hang	Set <code>HOPSWORKS_API_KEY</code> secret; guarded bootstrap with <code>st.error/st.stop</code> .
<code>PROJECT_ROOT</code> <code>NameError</code>	Reordered <code>utils.py</code> so the bootstrap and <code>PROJECT_ROOT</code> precede <code>PATHS</code> .
Python 3.14 / protobuf clash	Pinned Streamlit Cloud to Python 3.11.
Native build (<code>twofish</code>) failure	Added <code>build-essential</code> to <code>Dockerfile</code> and <code>packages.txt</code> .
“No forecast” / “native importance” on app	Ensured <code>forecast_3days.parquet</code> and <code>shap_importance.parquet</code> are in the bundle; re-boot clears the per-session cache.

Performance Analysis

Accuracy

On the chronological holdout, all four models generalize: Ridge $R^2 = 0.9788$, LightGBM 0.9431, RF 0.9339, XGBoost 0.9326, with single-digit MAE (1.78–3.06 $\mu\text{g}/\text{m}^3$). CV validation R^2 (Ridge 0.942, XGBoost 0.954, LightGBM 0.953) is consistent with the holdout, evidence that the validation protocol is honest. The champion (Ridge) has a CV gap of 0.0356 and a near-zero train–test gap (0.0035), confirming minimal overfitting.

Operational Performance

The daily training pipeline completes in roughly 2–3 minutes on a GitHub-hosted runner. Hourly ingestion is a single API round-trip plus an append. The retry session keeps both robust to transient API failures.

Cost

Open-Meteo is keyless and free; Hopsworks Serverless and Streamlit Cloud operate within free tiers; GitHub Actions minutes are modest given the short run-times. Continuous operation is therefore effectively zero marginal cost.

Limitations

- **Three months of history** cannot represent a full seasonal cycle; accuracy in unseen regimes (e.g. winter inversions) is unvalidated.
- **Feature-definition drift in the repo.** `create_features.py` no longer builds `pm25_roll_mean_3` or the `roll_std` features (leakage comment), yet the committed contract still selects `pm25_roll_mean_3`, `pm25_roll_std_3`, and `pm25_roll_std_6`. The forecaster rebuilds these on the fly, but the batch feature table and the contract are inconsistent and should be reconciled.
- **Gap-first selection optimizes for stability, not accuracy.** The criterion ranks by CV gap, so the champion may differ from the highest-accuracy model on a given holdout; in the latest run they coincide (Ridge), but that is not guaranteed.
- **Recursive error accumulation** grows uncertainty at later hours; predictions are point estimates without intervals.
- **NWP dependence:** forward accuracy is bounded by Open-Meteo's weather-forecast accuracy.
- **Selection instability** among near-tied models, mitigated operationally but not eliminated.
- **Single station / city scope;** no spatial modelling.
- **Alert delivery** is console-only; Slack/email is a TODO.

Future Improvements

1. Accumulate ≥ 12 months of data to cover seasonality and enable regime-aware validation.
2. Reconcile `create_features.py` with the feature contract (single source of truth for feature definitions).
3. Add **probabilistic forecasts** (quantile regression / conformal prediction) for calibrated uncertainty bands.
4. Introduce a **champion/challenger** promotion gate with explicit statistical tests to remove residual selection instability.
5. Activate the experimental `train_lstm.py` as a challenger once data volume supports it.
6. Wire `alerts.py` to Slack/email; trigger retraining *on drift* rather than purely on schedule.
7. Backfill richer NWP variables (humidity, wind, boundary-layer height) known to drive PM dispersion.

Lessons Learned

- **Pick the right diagnostic.** The R^2 gap, not the RMSE ratio, is the honest overfitting signal for high-variance models.
 - **Leakage is a design property.** Shift-before-roll and in-fold feature selection bake correctness in.
-

- **Contracts beat conventions.** A serialized `feature_config.json` permanently retired train/serve mismatch.
- **Statelessness simplifies operations.** A feature-store-centric design makes every runner reproducible and the dashboard host-agnostic.
- **Provenance must be visible.** The `local` vs. `version` tag turned an invisible race condition into a one-glance diagnosis.
- **Governance can be cheaper than code.** “One production uploader” fixed the champion-flip without changing the algorithm.
- **Keep artifacts and code in sync.** The `create_features/contract` mismatch shows how silently feature definitions can diverge from the models that consume them.

Conclusion

The Islamabad AQI Forecasting System demonstrates that a disciplined, overfitting-aware engineering approach can deliver a trustworthy, continuously operating PM_{2.5} forecaster even under a severe three-month data constraint. By making leakage prevention, an honest gap-based selection metric, a serialized feature contract, and a stateless feature-store-centric architecture into first-class design decisions, the system achieves strong, well-generalized accuracy (holdout R^2 from 0.93 to 0.98; champion CV gap 0.0356) while running autonomously — ingesting hourly, retraining daily, forecasting forward, monitoring drift, explaining predictions, and serving a public dashboard and API — at effectively zero marginal cost. The same blueprint generalizes to other cities, pollutants, and short-horizon environmental forecasting problems.

Bibliography

- [1] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
 - [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.
 - [3] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *NeurIPS*, 2017.
 - [4] A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
 - [5] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *NeurIPS*, 2017.
 - [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
 - [7] Open-Meteo, “Free Weather and Air-Quality API,” <https://open-meteo.com/>.
 - [8] Hopsworks, “Feature Store and Model Registry Documentation,” <https://docs.hopsworks.ai/>.
 - [9] GitHub, “GitHub Actions Documentation,” <https://docs.github.com/actions>.
 - [10] N. Siddiqi, *Credit Risk Scorecards*, Wiley, 2006. (Population Stability Index.)
 - [11] World Health Organization, “WHO Global Air Quality Guidelines,” 2021.
-

Unified Dependencies (requirements.txt)

```
1 pandas
2 numpy
3 scipy
4 scikit-learn
5 xgboost
6 lightgbm
7 joblib
8 pyarrow
9 requests
10 python-dotenv
11 hopsworks==4.7.*
12 confluent-kafka
13 matplotlib
14 # Dockerfile additionally installs: fastapi, uvicorn
15 # Streamlit Cloud provides streamlit at runtime; packages.txt:
    build-essential
```

Listing 10: Committed dependency manifest (single file for all environments; only hopsworks is pinned).

Daily Training Pipeline Stage Order

1. train_ridge (*critical*)
 2. train_random_forest (*critical*)
 3. train_xgboost (*critical*)
 4. train_lightgbm (*critical*)
 5. evaluate_models (*critical*) → model_metrics.json, best_model.json, best_model_metrics.json
 6. select_best_model (promote + write feature_config.json) (*critical*)
 7. forecast_future (*best-effort*)
 8. forecast_next_3_days (*best-effort*)
 9. data_drift (*best-effort*)
 10. model_drift (*best-effort*)
 11. alerts (*best-effort*)
 12. shap_explainer (*best-effort*)
 13. upload_model_to_registry (*best-effort*)
 14. upload_dashboard_bundle (*best-effort*)
-

AQI Category Mapping (pm25_to_category)

Table 11: PM_{2.5} to AQI category bands $[lo, hi)$, verbatim from `forecast_future.py`.

Band ($\mu g/m^3$)	Category	Marker
[0, 12)	Good	green
[12, 35)	Moderate	yellow
[35, 55)	Unhealthy (Sensitive)	orange
[55, 150)	Unhealthy	red
[150, 250)	Very Unhealthy	purple
[250, ∞)	Hazardous	maroon

Per-Fold Cross-Validation Detail

Table 12: Per-fold CV metrics from a representative training run (illustrative; absolute values shift run to run). Ratio = val RMSE / train RMSE.

Model	Fold	Train RMSE	Val RMSE	Train R^2	Val R^2	Ratio
Ridge	1 ($\alpha=10$)	2.6433	5.1975	0.9742	0.8424	1.97
	2 ($\alpha=0.01$)	2.1019	2.0642	0.9806	0.9724	0.98
	3 ($\alpha=0.01$)	2.0585	1.6350	0.9810	0.9735	0.79
Random Forest	1	3.4112	4.9877	0.9570	0.8548	1.46
	2	3.2275	3.8488	0.9543	0.9041	1.19
	3	3.1469	3.2044	0.9557	0.8982	1.02
XGBoost	1 (best_iter 1204)	1.6149	3.7903	0.9904	0.9162	2.35
	2 (best_iter 2477)	1.2480	2.2730	0.9932	0.9666	1.82
	3 (best_iter 3999)	1.0565	1.6149	0.9950	0.9741	1.53

— End of Report —